DSP in VLSI

Final Project

Eigen-value Decomposition by Iterative QR Operations

(Due by 06/17 05:00 PM)

In the final project, we will learn to use iterative QR operation for calculating eigen value decomposition and implement high throughput hardware for it. You can use all kinds of design techniques that you have learned in this course to achieve the good area- and processing time product. Upload all your programs (C/Matlab/Python/Verilog) and reports to NTUCool. Please put all your Verilog file/Synthesis results in one folder "FinalProject". TA will download it.

Two students form a group.

A. Algorithm

Assume that $\mathbf{A} = \mathbf{B}^{\boldsymbol{T}}\mathbf{B}$ is an $N \times N$ real matrix. We want to solve eigen value decomposition of matrix $\mathbf{A}$ to obtain its eigen value and eigen vector

$$\mathbf{A} = \mathbf{V}\boldsymbol{\Lambda}\mathbf{V}^{\boldsymbol{T}}. \tag{1}$$

where $\boldsymbol{\Lambda}$ is a diagonal matrix whose diagonal elements, λ_n, are eigen values, for $1 \leq n \leq N$.

$$\boldsymbol{\Lambda} = \begin{bmatrix} \lambda_1 & 0 & 0 & 0 & \cdots & 0 \\ 0 & \lambda_2 & 0 & 0 & \cdots & 0 \\ \vdots & & \ddots & & \vdots & \\ 0 & \cdots & 0 & \cdots & \lambda_{N-1} & 0 \\ 0 & \cdots & 0 & & \cdots & \lambda_N \end{bmatrix} \tag{2}$$

Denote eigen pair as $(\lambda_n, \mathbf{v_n})$, where $\mathbf{v_n}$ is the associated eigen-vector and is the nth column of $\mathbf{V}$.

The iterative QR algorithm for eigen value decomposition is given as follows.

Given symmetric matrix $\mathbf{A} \in \mathbb{R}^{N \times N}$
// First phase
$[\mathbf{U}_{\boldsymbol{EVD}}^{(\mathbf{0})}, \mathbf{A}^{(\mathbf{0})}]$=HessenbergReduction(A)
// Second phase

$\boldsymbol{i = 0,}$

1. **while (!converged)**
2. $\mathbf{T}^{(i)} = \mathbf{A}^{(i)} - \boldsymbol{\mu_i}\mathbf{I}$
3. $\left[\mathbf{Q}^{(i)}, \mathbf{R}^{(i)}\right] = \boldsymbol{QRD}(\mathbf{T}^{(i)})$
4. $\mathbf{T}^{(i+1)} = \mathbf{R}^{(i)}\mathbf{Q}^{(i)}$
5. $\mathbf{A}^{(i+1)} = \mathbf{T}^{(i+1)} + \boldsymbol{\mu_i}\mathbf{I}$
6. $\mathbf{U}_{\boldsymbol{EVD}}^{(i+1)} = \mathbf{U}_{\boldsymbol{EVD}}^{(i)}\mathbf{Q}^{(i)}$
7. $\boldsymbol{i = i + 1}$

End

Using a 4×4 matrix as an example, the operation of the Givens Rotation is as follows

(i) Consider $r_{1,1}$ and $r_{2,1}$. Use $r_{1,1}$ to eliminate $r_{2,1}$.

$$\begin{bmatrix} c^{(1)} & s^{(1)} & \mathbf{0} \\ -s^{(1)} & c^{(1)} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{1} \end{bmatrix}\begin{bmatrix} r_{1,1} & r_{1,2} & r_{1,3} \\ r_{2,1} & r_{2,2} & r_{2,3} \\ r_{3,1} & r_{3,2} & r_{3,3} \end{bmatrix} = \begin{bmatrix} r_{1,1}^{(1)} & r_{1,2}^{(1)} & r_{1,3}^{(1)} \\ 0 & r_{2,2}^{(1)} & r_{2,3}^{(1)} \\ r_{3,1} & r_{3,2} & r_{3,3} \end{bmatrix} \quad \textbf{(3)}$$

(ii) Consider $r_{1,1}$ and $r_{3,1}$. Use $r_{1,1}$to eliminate $r_{3,1}$.

$$\begin{bmatrix} c^{(2)} & \mathbf{0} & s^{(2)} \\ 0 & 1 & \mathbf{0} \\ -s^{(2)} & \mathbf{0} & c^{(2)} \end{bmatrix}\begin{bmatrix} r_{1,1}^{(1)} & r_{1,2}^{(1)} & r_{1,3}^{(1)} \\ 0 & r_{2,2}^{(1)} & r_{2,3}^{(1)} \\ r_{3,1} & r_{3,2} & r_{3,3} \end{bmatrix} = \begin{bmatrix} r_{1,1}^{(2)} & r_{1,2}^{(2)} & r_{1,3}^{(2)} \\ 0 & r_{2,2}^{(1)} & r_{2,3}^{(1)} \\ 0 & r_{3,2}^{(1)} & r_{3,3}^{(1)} \end{bmatrix} \quad \textbf{(4)}$$

(iii) Consider $r_{2,2}$ and $r_{3,2}$. Use $r_{2,2}$to eliminate $r_{3,2}$.

$$\begin{bmatrix} 1 & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & c^{(4)} & s^{(4)} \\ \mathbf{0} & -s^{(4)} & c^{(4)} \end{bmatrix}\begin{bmatrix} r_{1,1}^{(2)} & r_{1,2}^{(2)} & r_{1,3}^{(2)} \\ 0 & r_{2,2}^{(1)} & r_{2,3}^{(1)} \\ 0 & r_{3,2}^{(1)} & r_{3,3}^{(1)} \end{bmatrix} = \begin{bmatrix} r_{1,1}^{(2)} & r_{1,2}^{(2)} & r_{1,3}^{(2)} \\ 0 & r_{2,2}^{(2)} & r_{2,3}^{(2)} \\ 0 & 0 & r_{3,3}^{(2)} \end{bmatrix} \quad \textbf{(5)}$$

After three steps, three elements can be eliminated (set to zero), resulting in the upper triangular matrix $\mathbf{R}$. If the rotation matrix in each step is denoted as $\mathbf{G_i}$, for $i = 1{:}\,3$, then $\mathbf{Q}^T = \mathbf{G_3}\mathbf{G_2}\mathbf{G_1}$.

Givens rotation can be implemented by CORDIC. Taking the first step as an example, as illustrated in Figure 1. The values $r_{1,1}$, $r_{1,2}$, $r_{1,3}$ are sequentially fed into the X input of the CORDIC module, while the corresponding values $r_{2,1}$, $r_{2,2}$, $r_{2,3}$, are fed into the Y input. According to the Givens rotation algorithm (Equations (3) to (5)), the rotation angle must satisfy the condition,

$$-s^{(1)}r_{1,1} + c^{(1)}r_{2,1} = -\sin\left(\theta^{(1)}\right)r_{1,1} + \cos\left(\theta^{(1)}\right)r_{2,1} = 0\ ,$$

Namely, $\tan\left(\theta^{(1)}\right) = r_{2,1}/r_{1,1}$. Hence, the CORDIC module is programmed into vectoring mode when $r_{1,1}$ and $r_{2,1}$ enter so that $r_{2,1}$ can become 0 at the output. We can simply use 1 bit to record the clockwise or counterclockwise direction at each micro rotation stage. When $r_{1,m}$ and $r_{2,m}$ enter, for $m = 2,3$, the CORDIC module is program into rotation mode to complete the Givens rotations in (5) and (6), which can be further described by the following equation.

$$\begin{bmatrix} r_{1,m}^{(1)} \\ r_{2,m}^{(1)} \end{bmatrix} = \begin{bmatrix} \cos\left(\theta^{(1)}\right) & \sin\left(\theta^{(1)}\right) \\ -\sin\left(\theta^{(1)}\right) & \cos\left(\theta^{(1)}\right) \end{bmatrix} \begin{bmatrix} r_{1,m} \\ r_{2,m} \end{bmatrix} \tag{7}$$

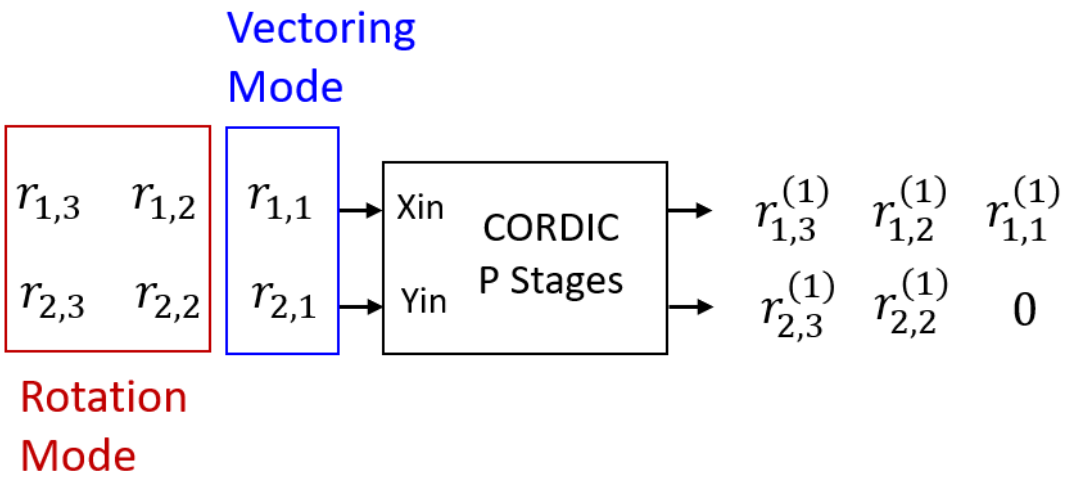

Fig. 1 Use CORDIC to realize the Givens rotation

B. Architecture

For high throughput applications, we can use systolic array to implement QR decomposition. The triangular systolic array for QR decomposition of 3×3 matrix is shown in Fig. 2. The west input of CORDIC is **Xin** and north input is **Yin**. The east output is **Xout** and the south output is **Yout.** When the input signal marked by the red rectangle enters from the west input, the CORDIC is switched into vectoring mode. Otherwise, the CORDIC is switched into rotation mode. Hence, the top left one executes step (i). The top right one compute step (ii). And the lower right one performs step (iv). Finally, the upper triangular matrix R appears at the output.

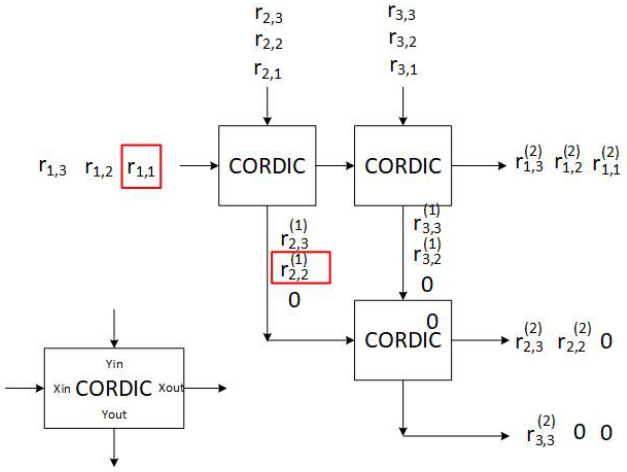

Fig. 2 The triangular systolic array for QR decomposition of 3×3 matrix.

Since the whole systolic array can be regarded as performing $\mathbf{Q}^T$, we can send identity matrix to obtain $\mathbf{Q}^T$ as shown in Fig. 3. Note that if you use more pipeline stages inside the CORDIC, the outputs become skewer and more D flip-flops are required to align the signals. A good tradeoff should be determined.

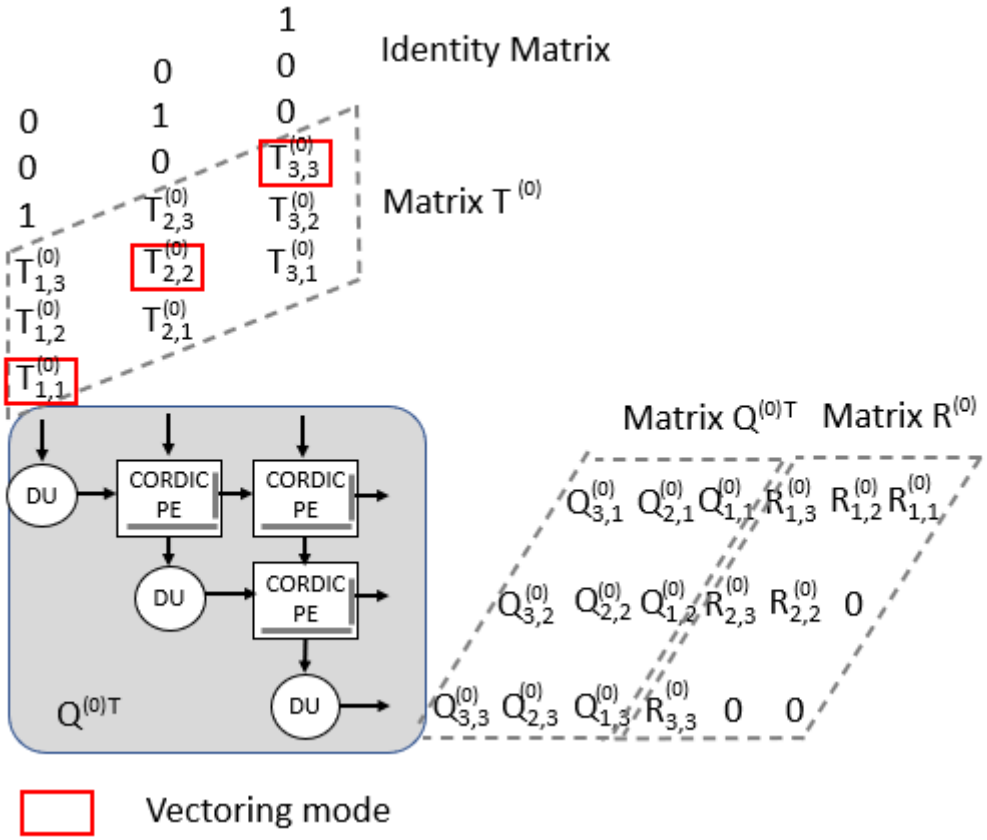

Fig. 3 Triangular systolic array for 3x3 QR decomposition.

In Step 4 of iterative QR algorithm, we need to perform $\mathbf{R}^{(i)}\mathbf{Q}^{(i)}$. Note that if the history

of the vectoring mode of $\mathbf{T}^{(i)}$ is recorded in the triangular systolic array, it performs left multiplication of $\mathbf{Q}^{(i)T}$. Thus if we send $\mathbf{R}^{(i)T}$ into the triangular systolic array, the output is $\mathbf{Q}^{(i)T}\mathbf{R}^{(i)T} = (\mathbf{R}^{(i)}\mathbf{Q}^{(i)})^{T}$, which is the transpose of $\mathbf{T}^{(i+1)}$. Thus, you can use the same hardware to perform the iterative QR decomposition. In addition, a buffer for matrix transpose is required, which has been illustrated in the lecture notes. The operation of $\mathbf{R}^{(0)}\mathbf{Q}^{(0)}$ is described in Fig. 4.

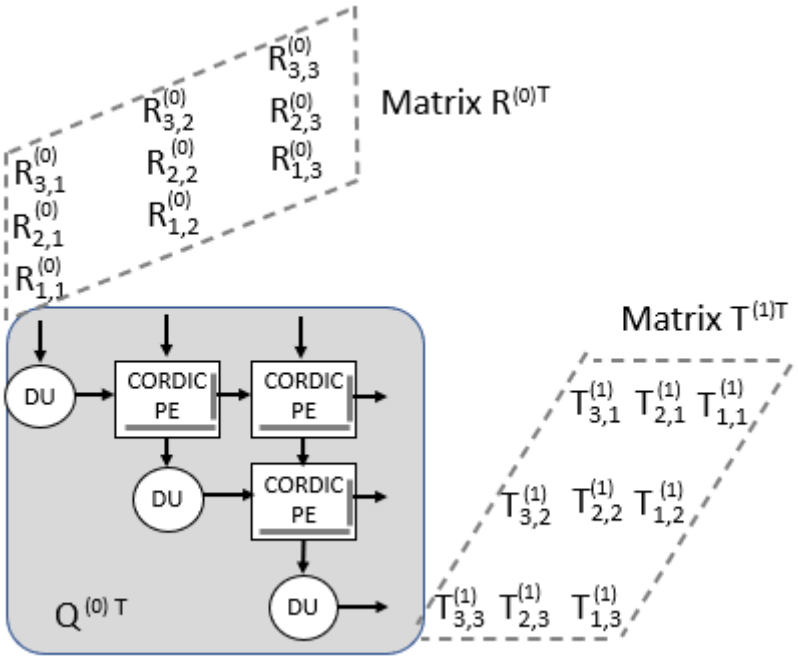

Fig. 4 Triangular systolic array with pipeline registers.

C. Test Patterns

To save your iteration times, 11 test patterns are generated and are given as Final11Pattern.mat. Its dimension is $3 \times 3 \times 11$ and contains 11 3×3 matrices. They are selected so that the iterative QR algorithm without shift can be completed within 4 iterations to achieve root-mean square error of 3 eigen values below 10^{-2}.

(a) Please use these 11 matrices to evaluate the word-length settings for QR decomposition.

(b) You can choose to use any shift or not.

(c) You can choose to deflate the matrix or not during the iterative operation.

(d) You can also choose to perform Hessenberg reduction or not at the beginning.

D. I/O Ports

For a fair comparison, the input signal is the original 3×3 matrices. Please use Matrix(:,:,8) as the test pattern in your hardware simulation. It has a moderate

convergence property.

The I/O of the hardware is defined as follows for fair comparison of area and throughput.

(a) **InValid**: If it is 1, the matrix **A** is sent through the **InData** ports. If it is 0, the **InData** ports are not processed. Note that this input signal is essential to control the switch between vectoring mode and rotation mode.

(b) **InData** ($3 \times W_I$): To accommodate the highest throughput, this port is mainly used for feeding matrix **A**. The word-length W_I is determined by yourselves.

(c) **OutData** ($3 \times W_O$): The computed eigenvalues and the associated eigenvectors.

(d) OutValid: If it is 1, the desired eigenvalues are sent out.

(e) **Clk:** clock signal

(f) **Reset:** reset signal

Requirements:

1. The word-length must be set so that the root mean square errors (RMSEs) of eigenvalues $\sqrt{\frac{1}{3}\sum_{i=1}^{3}(\hat{\lambda}_i - \lambda_i)^2}$ and eigenvectors $\sqrt{\frac{1}{9}\sum_{i=1}^{3}\|\hat{\mathbf{v}}_i - \mathbf{v}_i\|_2^2}$ are smaller than 10^{-2}, where λ_i and $\hat{\lambda}_i$ are the eigenvalues derived from the function call of eigenvalue decomposition of your software and your iterative algorithm; $\hat{\mathbf{v}}_i$ and $\mathbf{v}_i$ are eigenvectors derived from the function call and your iterative program. Show how you decide the wordlengths of CORDIC modules, the number of micro-rotation stages, the scaling factor. Draw the figure of RMSE versus wordlengths and versus the number of stages. (Note that you may need to use the "sort" command in order to check the difference of eigenvalues generated from different algorithm with arbitrary ordering of eigenvalues.)

2. Draw the respective RMSEs of eigenvalues and eigenvectors derived from your setting in Q1 versus set index (1~11) to construct your bit-true model. If RMSEs of P sets are larger than 10^{-2}, a deduction of $0.5P$ points will be applied. If more than 6 sets can not satisfy the criterion, the evaluation of AT product (area and processing time) is put in the last level.

3. Explain your hardware design concept in the report. Draw the block diagram of buffer for transpose in your design. Explain the control to perform the iterative operation.

4. Please show the hardware simulation results **with the input patterns quantized from Matrix(:,:,8).** All the settings must be the same as the answer you provided

in Q1. If the number of clock cycles in the simulation is large, then cut the whole timing diagram into several segments and paste them. Also, you need to keep and show the time stamp in your simulation figure for verifying the setting of the clock period. The timing diagram should be given like this.

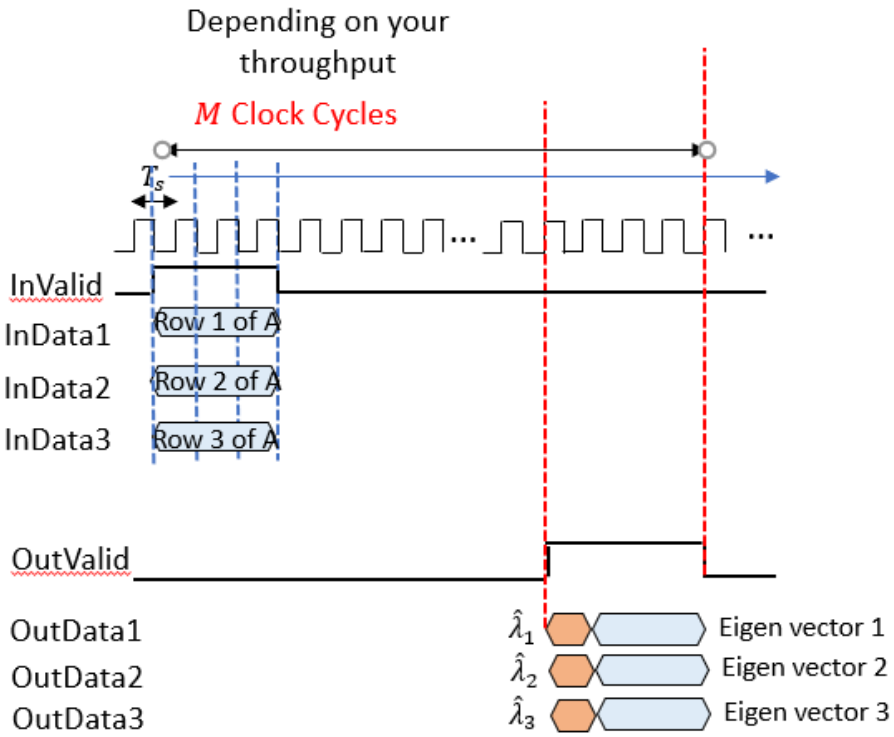

Fig. 5 Example of timing diagram

(a) Please provide the timing diagram of behavior simulation and calculate the RMSEs of the eigenvalues and eigenvectors. If the RMSE is larger than 10^{-2}, a deduction of 2 points will be applied. If the RMSE is larger than 10^{-1}, the evaluation of AT product (area and processing time) is put in the last level.

(b) Please provide the timing diagram of the post-synthesis simulation. You need to indicate the clock period setting (T_s) in your post-synthesis simulation and the values of M. The processing time is calculated by MT_s.

(c) Show that your post-synthesis simulation results are the same as the behavior simulation results.

5. Show that your implementation is consistent with your fixed-point simulation by drawing the figure indicating the errors between hardware output and fixed-point simulation output of the bit-true model.

6. Now calculate your AT product by

Area (or NA) × processing time (MT_s).
We will classify the AT product into 3~4 levels and evaluate the implementation results of your AT product
(Note: 15% of the score is decided by the level of the AT product. The remaining 80% is decided by the simulations, functionality, descriptions, and timing diagram. 5% is judged according to the innovations and improvements.)

7. Highlight any of your design innovations. List the working items and weightings of two members.

Reference:

[1] J. E. Guerrero-Ramírez, J. Velasco-Medina and J. C. Arce-Clavijo, "Hardware design of an eigensolver based on the QR method," *2013 IEEE 4th Latin American Symposium on Circuits and Systems (LASCAS)*, Cusco, Peru, 2013, pp. 1-4,